

Emuqu — A Measurement-Grounded Assistant with a Multilingual Regulated-Claim Perimeter

Typed fact catalog, runtime number verification, and one 17-language vocabulary enforced at input, output and build time

Technical White Paper • v1.3 • 8 October 2026

Chris Sharp • github.com/chrissharp80

Describes Emuqu (formerly Flow HRV, then Flow Recovery) at source revision e028039 (2026-10-07), scoring version v3.1.oct2026. Emuqu is a consumer wellness app, not a medical device.

Cite as: Sharp C. Emuqu — A Measurement-Grounded Assistant with a Multilingual Regulated-Claim Perimeter. Technical White Paper v1.3, 8 October 2026. github.com/chrissharp80/emuqu, docs/whitepapers/03-grounded-health-assistant.md.

Executive Summary

Emuqu is an iOS heart-rate-variability (HRV) and training app with a conversational coach ("Flo") that runs on six language-model providers: five hosted (Anthropic, OpenAI, Google Gemini, xAI Grok, DeepSeek) and one on-device (Apple Intelligence). The coach has two jobs beyond being helpful. First, it must quote the user's own measurements correctly: "your training stress balance (TSB) is -29" while the dashboard shows -15.5 is an error the user can check. Second, it must stay inside a regulated-claim perimeter in every shipped language. It must not name a condition for the user, assess heart rhythm, make treatment or prescription claims, or assert certainty about physiological state. It must also never leave a user without help when they report an emergency, an injury or thoughts of self-harm.

Emuqu handles both with deterministic, on-device mechanisms rather than instructions to the model:

1. **A typed fact catalog.** Every fact the model can read is one declarative entry that binds a key grammar, a description, a value type, an availability check and the resolver, so no resolver exists outside the catalog. Results share one JSON (JavaScript Object Notation) envelope with a closed, nine-value absence taxonomy across all six providers.
2. **A compact tool layer.** 21 polymorphic read tools and 18 allow-listed action tools, deduplicated and sorted for byte-stable schemas, with a capability index generated from the same list.
3. **Runtime number verification.** Claims about a metric's present value are checked against app state with per-metric tolerances, after spelled-out numbers are normalized. Only the claimed number is replaced, and the correction is fed into the next turn's prompt.

4. **Structural conflict resolution.** When the user contradicts cached numbers, the stale dashboard block is removed from the prompt rather than argued with.
5. **One multilingual regulated-claim lexicon.** 29 concepts in 17 languages drive three checks: a pre-send input guard that answers locally with fixed, localized replies; a sentence-level, negation-neutral output guard applied before display and speech; and a build-time linter over Swift source and every locale of the string catalogs. A continuous-integration (CI) gate checks, with one synthesized witness string per pattern, that the runtime lexicon covers the build-time list.

Section 5 states what is new as fourteen specific statements, each with the public work it builds on and its earliest dated record. The author developed the methods between April and October 2026; Section 8 gives commit-level dates. Section 6 lists the unit tests and CI gates that fix their behavior.

1. The Problem

1.1 Numbers that must match the dashboard

Language models produce fluent text that can contradict their inputs, the failure the hallucination literature calls intrinsic or input-conflicting hallucination [1]. In a health-data assistant it appears as checkable errors. In logs this app recorded, a model quoted training stress balance (TSB) values from -9 to -29 across turns while the dashboard showed one stable value, and in voice mode a model stated 72 bpm while the live heart rate (HR) was 91 bpm. The causes differ:

- **Guessing instead of retrieving,** from priors or an earlier turn.
- **Disagreeing sources,** such as a readiness snapshot frozen at workout start versus the current load.
- **Stale context:** a cached prompt block contradicts what the user just said. Models handle conflicting context inconsistently [2] and often fail to reject false premises [3], so a prose instruction ("trust the user") is not a reliable fix.
- **Spoken numbers:** in voice mode the model writes "negative thirty-one point seven", which a digit-matching check does not see.

Retrieval-augmented generation (RAG) [4] and tool use [5][6] reduce guessing. RARR [7] and FActScore [8] verify claims after generation with further model calls. Proof-Carrying Numbers (PCN) verifies claim-bound numeric tokens against an authoritative source under a tolerance policy, failing closed [9], and US Patent 12,670,149 detects and corrects inaccurate numeric comparisons in large language model (LLM) responses [10]. A streaming voice coach adds two requirements: the check must run in milliseconds on each sentence of free text before it is spoken, and it must read spelled-out numbers.

1.2 A regulated-claim perimeter in 17 languages

The app is a general-wellness product and not a medical device. The US Food and Drug Administration (FDA) general-wellness guidance [11] and App Store Review Guideline 1.4.1 [12] both turn on what the product claims. "You may have atrial fibrillation" is a claim the app cannot support; "see a doctor" without a crisis line fails a user in crisis. The perimeter must therefore:

- **not rest on the system prompt alone**, since prompt rules can be ignored, truncated or overridden by injection [13];
- **run on the device, between token arrival and speech**. Classifier guardrails such as Llama Guard [14] do not run there; Llama Guard 3 covers eight languages, with categories including Specialized Advice and Self-Harm [15]. Neither do rail frameworks such as NeMo Guardrails [16], whose streaming output rails run server-side [17];
- **work in all 17 languages** the assistant answers in;
- **not depend on negation detection**. NegEx [18] and its successors target clinical notes and are adapted language by language with per-language trigger lists [19], while negation is sentence-final in Japanese and Korean and split in French;
- **share one vocabulary between static copy and model output**.

2. Architecture Overview

Table 2.1 shows one user turn on the text path. The voice path is the same up to the provider call; after that, each speakable chunk goes through steps 10 and 11, in that order, before speech.

Table 2.1. Turn pipeline.

Step	Operation	Purpose
1	Trim input; reject empty	—
2	Input guard : classify the current message against the lexicon's refusal group in safety order	Answer emergencies, injuries, self-harm and rhythm questions locally; nothing is sent
3	Consent check for the active hosted provider (Appendix A)	No health data goes to an unconsented provider
4	History assembly: drop <code>localOnly</code> turns, truncate to the provider budget, summarize dropped turns	Guarded turns never leave the device later
5	Build or reuse the fact registry (the runtime form of the catalog); reset per-turn rate-limit counters	One address space for all facts
6	Compose tool list: 21 read tools + allow-listed actions, deduplicated and sorted	Byte-stable schema
7	Compose system prompt: stable prefix (rules, tool overlay, capability index) + variable sections (time, user-stated values, dashboard cache unless suppressed, live-workout marker, pending corrections, summary)	Grounding, conflict resolution, correction loop
8	Stream the provider round; tool calls go router → registry → envelope, at most 8 per turn	Retrieval with uniform absence semantics
9	Output guard, streaming : publish only complete sentences, each scrubbed	Sentences matching the lexicon are not displayed
10	Output guard, speech : scrub each utterance before text-to-speech (TTS)	No lexicon match is spoken
11	Number verification at each round end (chat) / per spoken chunk, after the speech scrub (voice)	Replace contradicted spans; record corrections

Step	Operation	Purpose
12	Output guard, final pass on the whole message	Covers text assembled outside the stream
—	Build time: copy linter over Swift and string catalogs; lexicon coverage gate	Static copy follows the same vocabulary

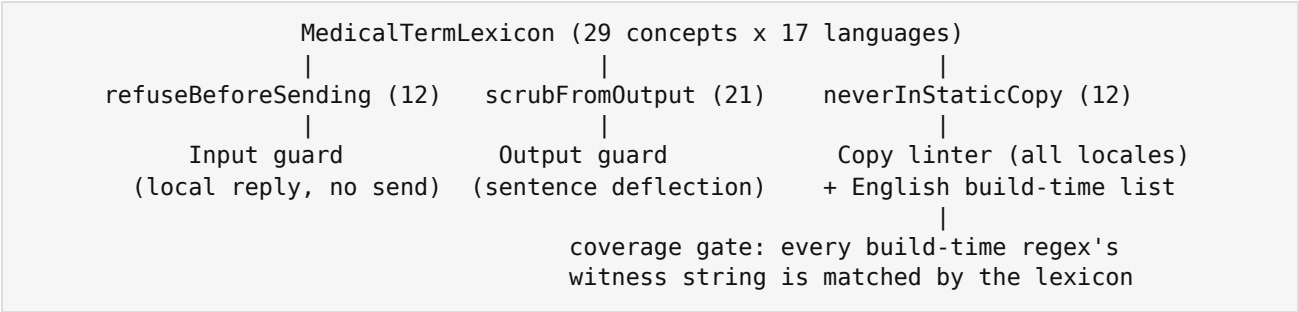


Figure 2.1. The shared vocabulary and its three enforcement points.

Why this shape. Each control sits where it has a deterministic answer. Retrieval is uniform, so a correct value is always available to quote. Verification checks the one class of claim with a single ground truth. The perimeter is one vocabulary applied wherever text crosses a boundary: into a provider, onto the screen, into the speaker and into the shipped bundle. None of the checks depends on the model following an instruction, so they behave identically for all six providers; the prompt sections (capability index, correction reminder) improve the next answer but are not relied on.

3. Methods

3.1 The typed fact catalog

A fact is a value of `FactEntry`, a type with exactly four cases (Table 3.1).

Table 3.1. Fact entry kinds.

Kind	Key	Resolver signature	Notes
fixed	literal dotted key, e.g. <code>user.profile.max_hr</code>	<code>() -> FactValue,</code> <code>.sync</code> or <code>.awaitable</code>	Carries <code>valueType</code> ("Int", "Double", "Duration", "Bool", "Record"...)
parameterized	pattern with one <code>\$param</code> , e.g. <code>session.by_date(\$date)</code> , plus an example argument	<code>(param, tail) -> FactValue</code>	Matches on leading tokens; trailing tokens pass as <code>tail</code>
composite	literal or parameterized	<code>(param?, registry) -> FactValue</code>	Declares <code>dependencies</code> (atomic keys only)

Kind	Key	Resolver signature	Notes
action	dotted verb key, e.g. <code>routes.library.rename</code>	<code>([String:String]) -> FactValue, .sync or .awaitable</code>	Declares named <code>ActionParams</code> with a required flag; description starts with <code>[ACTION]</code> by convention

Each entry also carries an `availability` closure (Section 3.1.4), which defaults to "always available" so a declaration is one expression. A namespace resolver exposes only `namespace: String` and `entries: [FactEntry]`. The registry derives the model-facing schema from `entries`, and the default resolve walks the same array to dispatch a key.

At e028039, 26 namespace resolvers are registered: 16 for health data, 9 for app and device state and actions, and a composites namespace registered last. Five share the head `app` and two share `workout`. They declare **238 entries**: 149 `fixed` synchronous, 8 `fixed` awaitable, 56 `parameterized`, 6 `composite` and 19 `action`.

Why it works. Because the resolver closure is a field of the declared entry, there is no second table to drift: a fact that can be resolved is a fact declared in the catalog, and the reverse. Adding a metric is one reviewed declaration that every provider sees on the next schema build.

3.1.1 Key grammar

```
key      := token ( "." token ) *
token    := name | name "(" argument ")"
```

1. Trim whitespace; reject empty input.
2. Scan characters, tracking parenthesis depth. A `.` at depth 0 ends a token. A `)` that would make the depth negative, a non-zero final depth, or an empty token (a `.`, leading dot) is an error.
3. A token ending in `)` and containing `(` splits into name and argument; an empty name is an error.

Dots inside arguments survive (`session.by_date(2026.04.21)` is valid).

Pattern matching. The pattern is parsed with the same grammar. Token by token, names must be equal. A pattern argument beginning with `$` captures the key's argument (a key with no argument fails); otherwise both arguments must be equal or both absent. The pattern may not be longer than the key, remaining key tokens become `tail`, and at least one capture must occur.

Dispatch. The registry tries every namespace whose `namespace` equals the key's head token, in registration order, then all other namespaces. This is how composites resolve under keys such as `user.profile.snapshot`. No match returns `missing(notRecorded, "no such key")`.

Tool names. Dots become underscores and a `($param)` section becomes a required string property: `session.by_date($date)` becomes `session_by_date` with `date`.

Why it works. One parse serves naming, matching and dispatch, so there is no translation layer between what the model asks for and what the resolver answers. A per-entry tool and a raw-key `lookup_fact` call reach the same

resolver, and `tail` lets one parameterized entry serve sub-keys without a declaration per field.

3.1.2 Value union and envelope

`FactValue` has nine cases: `integer`, `double`, `string`, `date`, `durationSec`, `boolean`, `missing(reason, detail?)`, `list`, `record`. Every result is serialized into one sorted-key JSON envelope:

```
{ "value": <typed JSON or null>,
  "missingReason": <reason, present only when absent>,
  "detail": <optional hint>,
  "asOf": <ISO-8601 with local UTC offset>,
  "confidence": "high" }
```

A non-finite double becomes `null` for that field only. Dates use the device's local offset, so the calendar day matches the `YYYY-MM-DD` arguments the tools accept. A nested `missing` in a list or record is rendered as an inline envelope, reporting absence per field. On success `missingReason` is omitted, so its presence is the absence signal. `asOf` is the resolution time and `confidence` is the constant `"high"`. The cloud tool loop and the on-device dispatcher for Apple's framework call the same serializer.

Provenance inside records. Source attribution lives in record fields where it is meaningful:

- `data_source`: `sport_wide` versus `route_specific` for cross-workout baselines, and `live_healthkit_pending_acceptance` for sleep and vitals read before the night is accepted. Tool descriptions tell the model how to word each, e.g. "compared to your typical run" versus "on this loop".
- `comparison_safe`: false when fewer than 2 prior samples back a baseline.
- Composite attribution (Section 3.1.5).

3.1.3 Absence taxonomy

Every resolver that cannot return a value picks one of nine reasons (a separate nine from the `FactValue` cases); there is no free-text-only absence.

Table 3.2. Absence reasons.

Reason	Meaning to the model
<code>notRecorded</code>	No data for this query (most common)
<code>notYetComputed</code>	Data exists; derived metric not computed yet. Retry later
<code>outOfRange</code>	Parameter outside the fact's valid range
<code>sensorDropout</code>	Signal too noisy or sparse to trust
<code>invalidParameter</code>	Malformed arguments; <code>detail</code> names the parameter and accepted range
<code>internalError</code>	Bug or budget overrun; report unavailable and stop
<code>tooMuchData</code>	Result over the size cap; narrow the query
<code>rateLimited</code>	Same call missed repeatedly this turn, or the turn's tool budget is spent

Reason	Meaning to the model
<code>partialData</code>	Composite partial result; composites carry it as <code>status: "partialData"</code> in their record (Section 3.1.5)

Why it works. Each reason maps to a different correct next action: retry later, fix the argument, narrow the query, stop, or tell the user nothing was recorded. A free-text error leaves the model to guess which applies; a closed enumeration makes it explicit and identical across providers. Per-field absence keeps a partly available record useful.

3.1.4 No-I/O availability gating

Each entry's `availability` closure returns `{hasData, validRange?, lastUpdated?}`. It must be synchronous and metadata-only (no HealthKit, file or network access); in practice it reads in-memory archive indexes and settings flags. The schema builder runs it once per build:

1. Drop every entry with `hasData == false` from the per-entry schema.
2. For a parameterized entry with a `validRange`, append `"Example: <example>. Data available as early as <Month YYYY>; do not request earlier dates."` to the parameter description. Only the month of the lower bound is given, with no rolling end date.

The workout-backed availability, for example, is `unavailable` until the archive holds one workout, then `valid` from the earliest to the latest archived workout date. The filtered per-entry schema feeds the action-tool list (Section 3.2) and the **catalog hash**: SHA-256 (Secure Hash Algorithm, 256-bit) over its sorted-key JSON, recomputed on each send. The hash detects drift in the per-entry descriptions and action schemas, the parts of the prompt that can change between sends. It changes only when an entry, a description or an availability result changes.

Why it works. Schema construction sits on the send path, so it must be fast and must not wait on I/O; metadata-only availability keeps it in memory and deterministic. Month granularity means a description changes at most once a month, which keeps the schema bytes stable for provider-side prompt caching [20] while still telling the model where data begins.

3.1.5 Composites with declared dependencies

A composite lists the atomic keys it reads. Composites may not depend on composites, and a test checks that every dependency names a real entry. On the asynchronous path:

1. Substitute the composite's captured parameter (if any) for `$name` in each dependency.
2. Await each distinct dependency through the async resolver (HealthKit or network children suspend here).
3. Merge results into a registry-held prefetch map, run the synchronous body (whose `registry.resolve(key)` reads the map first), then restore the previous map. Nothing suspends in between, so no other resolve sees the map.
4. The body returns `{present: [{key, value}], missing: [{key, reason, detail?}]}`, adding `status: "partialData"` when any child is missing.

A synchronous walk that reaches an un-prefetched awaitable child returns `missing(internalError, "resolves asynchronously ...")`; it neither blocks nor traps.

Why it works. Declared dependencies separate the slow part (awaiting children) from the logic (a synchronous body over known values), which keeps the body simple and testable. The scoped map cannot leak to concurrent resolves, and the `{present, missing}` record tells the model exactly which part of a summary is backed by data.

3.1.6 Registry rails

Five rails apply to every call. Each answers in the same envelope.

1. **Required action arguments** (before the action runs). A missing one returns `invalidParameter` naming it; unknown keys are dropped.
2. **Numeric range checks** (when a resolver parses its arguments, before it reads data). Each numeric parameter is declared once with a closed range, an optional excluded lower bound and an integer flag. Only a finite in-range number (whole where flagged) is accepted. Anything else, including "inf", "nan" and "1e20", returns `invalidParameter` stating the requirement, e.g. "ordinal must be a whole number from 0 to 100000, got '-1'". Comma-separated blobs must have exactly the expected field count, empty fields counted (60, ,5 has three).
3. **Time budget** (during resolution). On the per-entry path, resolution over 2.0 s is discarded as `missing(internalError, "resolver timed out...")`; over 0.5 s logs a warning, and over 4.0 s on an action logs a watchdog warning. On the compact read-tool path the budget only logs, because legitimate HealthKit and network waits can exceed 2 s.
4. **Repeat-miss limit** (on the result). The signature is `toolName + "::" + argsJSON`. Each *missing* result increments a per-turn counter; above 2, the result becomes `missing(rateLimited, "Same tool+args returned missing 2 times this turn. Stop retrying...")`. Successful results do not count, and counters reset each user turn.
5. **Size cap** (on the result). A non-missing result whose envelope exceeds 80,000 UTF-8 bytes becomes `missing(tooMuchData, "... Ask a narrower query...")`.

Why it works. Every rail answers in the same envelope, so the model is steered rather than stopped: it stops retrying, narrows a query or fixes an argument, and the turn still ends in a useful reply. Counting only misses per exact signature targets the loop that wastes a turn without penalizing legitimate repeated reads. Stating the accepted range in words lets the model correct the call on the next round.

3.2 Compact tool layer

The model sees 39 tools, not 238:

- **21 read tools** with fixed names and string parameters, most taking a selector (`which`, `field`, `metric` or `aspect`) that routes to catalog keys: `get_today`, `get_session`, `get_recovery`, `get_hrv`, `get_sleep`, `get_vitals`, `get_workout`, `list_workouts`, `get_walks`, `get_baseline`, `get_user`, `get_app_state`, `get_training_load`, `get_score_meta`, `get_breadcrumbs`, `get_routes`, `get_tags`, `get_workout_live`, `get_assistant_state`, `get_healthkit`, and `lookup_fact` for any raw catalog key.

- **18 action tools** under their catalog names (e.g. `assistant_email_compose`, `routes_library_rename`, `web_search`, `directions_routeTo`), taken from the availability-filtered per-entry schema through an explicit allow-list. Of the 19 declared actions, `location.roads_ahead` is not allow-listed.

Read tools are appended first, then actions. Duplicates by name are dropped (first wins, with a warning, because one provider rejects duplicates), and the list is sorted by name. Action schemas list `required` sorted; every property is a string and the resolver converts types.

Capability index. The read-tool list also generates a system-prompt section: each tool's name and the first sentence of its description, capped at 170 characters, with an instruction to call the matching tool before answering any question about the user's own data.

Ranking. A BM25 ranker [21] ($k_1 = 1.5$, $b = 0.75$) filters the schema only above 40 tools, so at 39 it passes the list unchanged. The on-device provider ranks tools with the same scorer and keeps the highest-ranked ones that fit its context token budget.

Per-turn budget. When `callsSoFar + callsThisRound > 8`, every call in that round gets `missing(rateLimited, "tool budget exceeded for this turn")`. The model has one more round to answer in text, and further tool calls are dropped. The on-device dispatcher enforces the same constant.

Why it works. A few polymorphic read tools are easier for every provider to choose among than hundreds of narrow ones, and they fit the on-device context. Catalog names keep every side effect identifiable in audit logs. Sorting and deduplication make the schema bytes a pure function of the catalog, so the stable prefix caches well, and generating the index from the same list means the prompt always describes the tools that exist.

3.3 Runtime number verification

3.3.1 Claim shapes and tolerances

Two families of metrics are checked (Table 3.3). ATL and CTL are acute and chronic training load, ACWR is the acute:chronic workload ratio, RMSSD is the root mean square of successive differences of beat-to-beat intervals, and DFA α_1 is the short-term scaling exponent of detrended fluctuation analysis.

Table 3.3. Verified metric families.

Family	Metrics	Source of truth	Where applied
Live workout	HR, power, DFA α_1 , HR drift	Live workout snapshot	Voice, each chunk before speech
App state	TSB, ATL, CTL, ACWR, RMSSD, recovery score, last night's sleep duration and efficiency	Training-load registry (same accessor as dashboard and tools); latest reliable overnight session; latest overnight with sleep	Chat, every tool-loop round end; voice, each chunk before speech

Table 3.4. Claim shapes and tolerances.

Metric	Claim shape (case-insensitive, simplified)	Tolerance	Replacement
HR	"your [current/live] HR/heart rate [is/s]..." or "HR right now/currently/now", filler, $\backslash d\{2,3\}$ bpm	5 bpm	N bpm
Power	same shape, "power/wattage", $\backslash d\{2,4\}$ W/watts	15 W	N W
DFA $\alpha 1$	same shape, " $\alpha 1$ /alpha 1", optional "DFA", decimal only	0.10	%.2f
HR drift	same shape, "[HR] drift", number + %	1.5 percentage points	%.1f%
TSB	TSB + optional "negative" + signed number	2.0	%+.1f
ATL / CTL	label + number	3.0	%.1f
ACWR	label + number	0.15	%.2f
RMSSD	RMSSD [of] N [ms]	8.0 ms	%.0f ms
Recovery	recovery N	4.0 on 0–100; a claim ≤ 10 within 1.0 of the 0–10 score is accepted	on the claim's scale
Sleep duration	"slept [for] [about] Nh [M m]", "sleep [duration] was ...", "... of sleep"	5 min if minutes or decimal hours given; 30 min for whole hours	"6h 55m" or "6 hours and 55 minutes", matching the claim
Sleep efficiency	"[sleep] efficiency [of/was/...] N%", "N% [sleep] efficient/efficiency", in a sentence mentioning sleep	1.0 percentage point	N%

The TSB, ATL, CTL, ACWR, RMSSD and recovery patterns reject a number followed by a time or count unit (days, h, weeks, nights, minutes, sessions, workouts), by %, or by more digits, so "CTL 42-day window" is not a claim.

3.3.2 Algorithm (app-state family)

1. **Normalize spelled numbers.** Match $\backslash b(\text{negative}\backslash s+)?(\text{ones-or-teen} \mid \text{tens}([\backslash s]+\text{digit})?) (\backslash s+\text{point}\backslash s+\text{digit})?\backslash b$, alternations longest first, and replace with digits ("negative thirty-one point seven" $\rightarrow$ -31.7), recording (originalRange, normalizedRange) rewrites. Range 0–99, at most one decimal digit, English words.
2. Run the metric regexes on the normalized text. Each names a **value** group and a **claim** group (number plus unit, the span to replace); the metric label is never in the replaced span.
3. Find each match's sentence (the enumerator does not split "17.2"). Discard it if the sentence has a **time marker** (ago, yesterday, last week/month/.../weekday, previous(ly), earlier, before, on) or a **target/comparison marker** (average, mean, baseline, typical, usual, target, goal, keep, stay, peak, max, min, trend, would, could, if, should, threshold, under, below, above, over, between, range, higher, lower, than, and similar).
4. Negate a "negative N" claim. Keep the match only if $|\text{claimed} - \text{actual}|$ exceeds the tolerance.

5. Recovery: drop a claim ≤ 10 if $|\text{claimed} - \text{score}_{10}| \leq 1.0$, where score_{10} is the score on the 0–10 scale; report a remaining claim ≤ 10 on that scale.
6. Sleep: drop matches in sentences with general-norm markers ("need", "recommended", "adults", "per night", ranges such as "7-9") or other-night markers (month and weekday names, "nights", "weeks", dates); drop overlaps.
7. **Remap** ranges to the original text. An offset inside a rewritten span snaps to the rewrite's original start or end, so "negative thirty-one point seven" is replaced whole.
8. Replace claimed spans back-to-front. Every other word, including unrelated spelled numbers ("two sessions"), is unchanged.

The live-workout family runs steps 2–4 without normalization and requires the "your X is" / "X right now" shape, because a bare "150 bpm" is usually a target or a different metric.

3.3.3 Where verification runs

In chat, verification runs at the end of every tool-loop round, including rounds that end in a tool call. It runs before the text that preceded the tool call is cleared from the visible reply, and the corrected text replaces the saved turn. In voice, each chunk passes output scrub → live-workout verification → app-state verification → preamble → speech normalization, so the corrected number is the one spoken.

Why it works. A claim about the value *now* has exactly one ground truth in app state, so it can be checked deterministically. History, averages, targets and norms do not have one; the markers keep them out of scope, so the verifier corrects only what it can know. Tolerances follow dashboard rounding and each metric's noise, so rounded or approximate statements stand. Replacing only the number-and-unit span keeps the model's sentence and reasoning intact. Normalizing spelled numbers and remapping offsets let one set of regexes serve chat and voice. The dual-scale recovery rule and precision-matched sleep tolerance accept the ways people naturally state those values.

3.3.4 Correction loop

Each discrepancy enters a process-wide, lock-guarded buffer, deduplicated by (metric, claimed, actual) (voice sees the same claim in the spoken chunk and the saved turn) and capped at the 4 most recent. The next system prompt reads and clears it into a variable section outside the cached prefix:

```
# Last turn correction – DO NOT FABRICATE
Your previous response contained numbers that contradicted live data. The saved reply was
corrected, but the user may already have read or heard the wrong number – if it matters,
correct it briefly. For any of these metrics next turn, CALL the appropriate tool (...)
and
quote what comes back – never estimate or interpolate.
• You said TSB=-29.0 – actual was -15.5.
```

Why it works. Each discrepancy yields one reminder line on the next turn, naming the metric, both values and the tool to call, so the next answer is grounded without regenerating the current one. Consume-once and the cap keep the prompt short, and placing the section outside the prefix keeps the cache intact.

3.4 Structural resolution of user-asserted values

Before the variable prompt sections are built, the user-assertion detector takes the last 6 turns of the conversation, keeps the user turns that are not `localOnly`, and reads them newest first. For each message:

1. Normalize spelled numbers (Section 3.3.2, step 1).
2. Remove every sentence ending in `?` or `?`; questions assert nothing.
3. For each known metric name, find the *last* `\bmetric\b\s*[connector]\s*number`. The names cover training-load, HRV, heart-rate, recovery, sleep, workout and vitals terms, tried longest first so "max hr 190" is not read as "hr 190". The connector is optional except for `sleep` and `recovery`, which need one of `is/of/=/:/was/being`. Clock times (`5 am`, `11 pm`, `5:30`, `o'clock`) are excluded.
4. Blank each match after use, so the newest assertion per metric wins. Transcription variants (`bsb`, `tsp`, `tbs` → `tsb`, ...) fold into the canonical key only if the canonical spelling gave no value.
5. Set `dashboardContradicted` on a contradiction phrase ("the dashboard says", "real numbers are", "you're wrong", "wrong number", ...) or on a sentence opening "no/actually, it's/it is" followed by a number or a word such as `not/different/wrong/higher/lower/more/less`.
6. Set `explicitOverrideRequested` on phrases such as "stop calling tools", "use these numbers", "I don't care what the dashboard...", "stop saying".

Policy: if any signal is present, inject `# User-stated values (authoritative – fresher than any cache)` with one `- key: value` line per assertion (training-load metrics first), adding a stale-cache sentence on contradiction or override. If `dashboardContradicted` or `explicitOverrideRequested` is set, **omit the dashboard training-load block entirely**.

Why it works. A model given two conflicting numbers and an instruction to prefer one still holds both, and may average them or pick the wrong one. Removing the stale block leaves nothing to reconcile. Reading statements only, newest first, with a required connector for ambiguous words, keeps questions and clock times from being taken as assertions.

3.5 The multilingual regulated-claim lexicon

Concept structure. A concept has an `id`, a `latin` list of word-bounded alternatives and an `unbounded` list of substring alternatives, compiled as

```
(?: \b(?:latin_1|latin_2|...)\b | (?:unbounded_1|unbounded_2|...) )
```

case-insensitively against the original text (never a case- or diacritic-folded copy), so match ranges align with the string being rewritten. Accented forms are spelled out (`m[ée]decin`). Latin-script and Cyrillic alternatives go in the `latin` list. CJK (Chinese, Japanese and Korean) and Arabic alternatives go in the `unbounded` list and match as substrings, because CJK text has no inter-word boundary (a `\b` around `心房細動` never matches in running Japanese).

Languages. English plus `ar`, `da`, `de`, `es`, `fi`, `fr`, `is`, `it`, `ja`, `ko`, `nb`, `nl`, `pt-BR`, `ru`, `sv` and `zh-Hans`: 17. A test fails if any concept lacks a Chinese or Japanese, a Korean, or an Arabic alternative in its `unbounded` list.

Concepts (29).

- Rhythm: atrial fibrillation, arrhythmia, irregular heartbeat.
- Conditions: named cardiac condition, acute cardiac event, neurovascular event.
- Claims: diagnosis, cure, treatment claim, prescription, regulatory clearance, pathology, clinical physiology labels, symptom of disease, speculative diagnosis.
- Risk framing: injury risk, overtraining, risk-zone framing.
- Certainty: categorical autonomic state, physiological certainty, unsupported metric verdict.
- Referral: medical referral.
- Input-side: chest pain, breathlessness, syncope, severe symptom, danger judgement, self-harm, injury.

A separate two-concept offensive-language list uses the same structure.

Benign-homonym exclusions are written per alternative, because the qualifier's position differs by language (Table 3.5).

Table 3.5. Benign-homonym exclusions.

Benign sense	Exclusion
Respiratory sinus arrhythmia (normal physiology)	EN (?<!sinus\s)arrhythmias?; FR arythmies?(?!s+sinusales?); ES/PT aritmias?(?!s+sinusal(?:es)?); NL/DE/DA/NB/FI (?<!sinus)...; RU (?<!синусовая\s)(?!синусовой\s)аритми[яи]; JA (?<!洞性)不整脈; ZH (?<!窦性)心律失常; KO (?<!동성)(?!동성)부정맥; AR negative lookahead for التنفسي / الجيبي . German Sinusarrhythmie passes because the compound has no inner \b
Software diagnostics ("Diagnostic log")	diagnos... (?!s+(?:log logs data tool tools bundle)), likewise on bare diagnose and Spanish/Portuguese forms
Swimming "stroke rate", "stroke of luck"	no bare stroke ; only (?:having had have)\s+a\s+stroke(?:\s+of\s), "stroke symptoms", "mini-stroke"
"Suicide sprints" (a drill)	su[īi]cide (?!s+(?:sprints? runs? drills? shuttles? lines?)\b)
Finnish <i>parantaa</i> ("improve"), Italian <i>prenditi cura</i> ("take care"), Icelandic <i>greining</i> ("analysis"), Germanic <i>Rezept/recept</i> ("recipe")	only disease-object or prescription-only shapes match
"This is your recovery score"	(?-i:[Tt]his\s+IS\s+your) is case-sensitive inside a case-insensitive pattern, so only the emphatic claim matches

Groups.

- **refuseBeforeSending** (12) = rhythm (3) + emergency (7: chest pain, breathlessness, syncope, severe symptom, self-harm, acute cardiac event, neurovascular event; the input guard tests self-harm first and answers it with the crisis reply) + danger judgement + injury.
- **scrubFromOutput** (21): all rhythm, claim, risk, certainty and referral concepts, plus named cardiac condition and neurovascular event.

- `neverInStaticCopy` (12): atrial fibrillation, arrhythmia, the three condition concepts, regulatory clearance, cure, prescription, treatment claim, speculative diagnosis, pathology and clinical physiology labels.

Eight concepts are **input-only**:

- Chest pain, breathlessness, syncope, severe symptom, self-harm and injury, because the model must be able to say these words when explaining why the app cannot help.
- Danger judgement, because its bare adjectives (`dangerous`, `gefährlich`) appear in the app's own negating educational copy ("not because the number is dangerous in itself"). Assertion-shaped danger framing is covered on output by risk-zone framing.
- Acute cardiac event, a strict subset of the already-scrubbed named cardiac condition. It exists so the input guard can refuse "am I having a heart attack?" without refusing "is my resting HR bradycardia?", an ordinary endurance-athlete question.

Every concept is in the input group, the output group, or both (rhythm and neurovascular event).

`neverInStaticCopy` excludes concepts the app's copy uses on purpose. The localized redirect "Consult a doctor if consistently low" matches `medicalReferral`, and the whole lexicon produced 45 such matches over the string catalogs. The 12 selected concepts matched nothing in any locale, so the group can act as a hard gate.

Why it works. One vocabulary consumed at three points means a term forbidden in shipped copy is also handled at input and in model output, in every language, and one edit reaches all three. Splitting by script gives each language the boundary rule that fits it, and matching on the original text keeps replacement ranges exact. Per-language exclusions at the qualifier's natural position let normal physiology, software terms and sports vocabulary through. Per-concept group membership lets each enforcement point act without rewriting the app's own safety language.

3.6 Input guard

Prompt rules. The system prompt carries four medical-boundary rules, which the guards back up deterministically: A, no personal diagnosis; B, fixed replies for severe red-flag symptoms, self-harm and injury; C, rhythm questions answered with the app's limits and a pointer to a clinically validated electrocardiogram (ECG) feature; D, medications discussed only as general information, with no dosing or personal prescription.

Order (first match wins): 1. self-harm → crisis reply; 2. emergency → symptom reply; 3. injury → injury reply; 4. rhythm → rhythm reply; 5. danger judgement → symptom reply; 6. none → proceed.

Replies are fixed catalog strings, localized into all 17 languages:

- Crisis: an apology, "please reach out to someone now", the local emergency number, findahelpline.com, and 988 for the US. No mention of a doctor.
- Symptom: "If you're exercising, stop now. Talk to your doctor about that. I can't assess your health — Emuqu is a fitness coaching app, not a medical device. If you're feeling unwell, please contact a clinician (or your local emergency number for severe symptoms)."
- Injury: stop, call the local emergency number if serious, the app cannot assess injuries, otherwise see a clinician.
- Rhythm: the app does not detect atrial fibrillation (AFib) or arrhythmia and is not a medical device; talk to your doctor; a clinically validated ECG feature exists for that purpose.

On a refusal, the guard (1) appends the user turn and the canned reply, both flagged `localOnly`, and persists them; and (2) returns without calling any provider, including the on-device one, and before the consent check. The flag lives on the stored turn. History assembly withholds such turns before budgeting (they are never counted as "dropped"), summarization filters them again, and the user-assertion detector skips them. A guarded message therefore cannot reach a provider later as history.

Feature flag. The guard defaults on; the self-harm branch runs even when it is off: `if flagOn || classify(text) == selfHarm`.

Prompt parity. Rule B asks the model to reply exactly with the symptom, self-harm and injury texts. Unit tests check that the symptom and injury replies appear verbatim (after whitespace collapse) in both the cloud and on-device prompts.

Voice echo trap. The spoken rhythm refusal itself contains "AFib" and "arrhythmia". A transcript is therefore dropped as an echo when it would trigger the refusal that is already the last assistant turn and at least 40% of its tokens overlap that reply.

Why it works. An emergency, injury or self-harm message needs a correct answer at once and identically every time, so a fixed translated string answers it, not a model. The order puts the most urgent resource first. Self-harm leads because its reply must carry a crisis line rather than a doctor referral. An acute symptom outranks rhythm because its reply names an emergency number, so "skipped beats and chest tightness" does not get the ECG pointer. A danger-judgement request ranks below rhythm because "should I be worried about my AFib" is best served by the ECG pointer. Running before consent and flagging the stored turn keep the message on the device now and later. Prompt parity gives the user the same words if a message reaches the model, for example with the guard flag off.

3.7 Output guard

Scope. Sentences are deflected when they name a condition for the user, speculate about one, assess rhythm, claim cure, treatment, prescription or regulatory clearance, frame risk as an injury prediction or "danger zone", assert categorical autonomic certainty, or rank a metric as truth. General physiology passes: HRV, parasympathetic activity, respiratory sinus arrhythmia, training load, sleep. Prompt rules A and D tell the model not to give personal diagnoses, dosing or personal prescriptions; on output, prescription vocabulary is matched.

Negation-neutral deflections. Each deflection is true whether the replaced sentence affirmed or denied the term, e.g. "Emuqu measures beat-to-beat timing; it doesn't assess heart rhythm. A clinician or a clinically validated ECG is the right place for that question." Deflections are catalog strings in the in-app language, and a test checks that none trips any rule. There are 13 medical deflections (9 concept-specific, 4 shared: rhythm, load, out-of-scope, observation fallback) plus one offensive-language line.

Offensive-language entries. One offensive-language concept runs on every reply. A second, English-only concept holds entries that are ordinary words or names in another shipped language. It applies only when the reply is recognized as English: the language recognizer's top hypothesis must reach a confidence of 0.6 on a text of at least 3 words, and otherwise the in-app language decides.

```

scrub(text):
    active = medicalRules + offensiveRule (+ englishOnlyRule when it matches and the reply
is English)
    r = firstMatch(text, active)
    if text empty or r is none: return text unchanged
    segments = split(text) so that concat(segments) == text exactly
    rebuilt = ""
    for each segment:
        core = segment without leading/trailing whitespace
        rule = firstMatch(core, active)
        if rule: rebuilt += leadingWS + rule.deflection + trailingWS; record (rule.reason,
core)
        else: rebuilt += segment # byte-identical
    if nothing was replaced: # the match straddles a terminator
        m = r.regex.firstMatch(text)
        straddled = text with every segment overlapping m collapsed into ONE r.deflection
        if m found and firstMatch(straddled, active) is none: return tidy(straddled)
        return r.deflection # whole message
    return tidy(rebuilt) # " "->" ", " "->".", " "->","

firstMatch(t, rules):
    r = first rule whose regex matches t
    if r: return r
    plain = t with every '*', '_', '`' removed
    return plain != t ? first rule matching plain : none

```

Segmentation. A segment runs through a terminator from `{. ! ? \n . ! ? , ? ...}`, any further terminators (`?! , ...`), then whitespace. A `.` after a digit is a decimal point when a digit follows; at the end of the text it is undecided and does not split. A sentence is rewritten at most once and rewritten text is never re-scanned, so the pass is linear with no iteration cap. The markdown retry catches emphasis inside a term (You may have **atrial* fibrillation.*). A caller that needs the match's range re-runs the regex on the original text and deflects the whole message if it cannot locate it.

Where it runs.

1. **Streaming buffer** (chat and the voice cursor). Deltas publish at most every 33 ms. Each publish splits at the last terminator, scrubs only the complete part and holds the tail back, so a matched phrase does not render, even partially. The voice "speakable cursor" advances only over published text. At round end and on every exit path, including errors and Stop, the tail is force-published, scrubbed.
2. **Speech.** Every utterance (streamed chunks, completed turns, error lines, wake-word acknowledgements) passes one function, scrub → number verification → preamble, before the synthesizer.
3. **Final pass** over the whole message after the stream ends.

Why it works. A denial is a finding too ("This isn't AFib" rules a condition out), and negation is spread across sentences in ways one pattern cannot follow in 17 languages (sentence-final in Japanese and Korean, split in French, a negative verb in Finnish). Deflections true in both readings remove the need to decide which reading applies, so the replacement is true in either reading, in every language, without a negation model. Whole-sentence replacement keeps replies grammatical, and holding back tails keeps a matched phrase off screen even for a frame. One speech function leaves no path to the speaker that skips the guard, and the straddle rule makes the boundary case fail closed.

3.8 Build-time copy linter

The linter scans the app source tree, the Watch app and both string catalogs:

1. **Swift literal scanner.** One pass handles `//` comments, *nested* `/* */` comments, `"""` multi-line literals and raw literals (`#"..."#`, `##"..."##`, where backslash is not an escape), blanking interpolations `\(expr)`. Measured on this code base, a quote-pairing regex missed 93% of the characters inside multi-line literals, including all long-form help articles; the scanner reads them.
2. **Prohibition-context exemption.** The system prompt quotes forbidden framings to forbid them ("Never tell the user that a number 'predicts injury'..."). A match is excused only in two cases. (a) A guard such as `never|don't|do not + tell|say|use|describe|call|label|refer|frame|claim|imply` matches in the **comment-stripped** source within a 3-line window (the match line and the 2 lines above), and the literal's opening fragment (its first 40 characters, at least 3) lies in that same sentence. (b) The match's own sentence inside the literal contains a guard. A comment cannot excuse a string, and catalog values are never excused this way.
3. **Catalogs.** An allow-listed catalog *key* clears its translations too. Every catalog is checked against the English build-time list (62 patterns) and, in *every locale*, against the multilingual `neverInStaticCopy` concepts. The linter parses these from the Swift lexicon source (bracket-matched `latin` and `unbounded` arrays, unescaped one level, compiled with the runtime's `\b` rules).

Each of the following exits with code 2: a missing scan path, an unreadable file, an unparseable catalog, a missing `neverInStaticCopy` group or zero usable concepts, or an uncompileable pattern. PDF reports and the post-workout Coach Report email body are built from templates, contain no model-generated text, and are governed by this linter; model-drafted emails are a separate path (Section 7).

Why it works. Static copy ships in the bundle and never passes the runtime guards, so it is checked where it is written. Scanning literals as the compiler reads them covers long-form text, not just one-line strings. Scoping the exemption to the prohibiting sentence on comment-stripped source lets the prompt name what it forbids without letting a nearby comment excuse other copy. Reading the multilingual set from the runtime source means every locale follows the model's vocabulary with no second copy to keep in step.

3.9 Coverage gate

The CI gate checks that the runtime lexicon recognizes a concrete example of every build-time pattern:

1. Extract every literal from every `latin:/unbounded:` array in all lexicon files (a glob, so a file split hides nothing). Bracket matching skips string literals, so a `]` inside a character class does not close the array. Compile each, word-bounding `latin` alternatives.
2. **Fail (exit 2)** if any alternative matches the empty string or fewer than 100 were extracted; either means the parser is broken. At e028039 there are 985.
3. For each build-time pattern, **synthesize a witness:** remove `(?i)` and lookarounds; `\s+` → space; remove `\s*` and `\b`; reduce character classes to one member (`\s` → space, `\x` → `x`); drop leftover quantifiers; repeatedly reduce `(a|b)?` → `""`, `(a|b)` → `a`, `(a)?` → `""`, `(a)` → `a`; remove `?`. A pattern whose witness reduces to the empty string is skipped and not counted; at e028039 all 62 patterns produce a witness.
4. **Fail (exit 2)** if the witness does not match its own pattern.

5. **Fail (exit 1)** if no lexicon alternative matches the witness, listing each gap.

Why it works. Two hand-kept vocabularies drift. A witness turns each build-time pattern into a concrete example the runtime lexicon must recognize, so a term added to only one list fails CI. The self-check and the vacuity guards ensure the gate cannot pass because its own parser broke.

4. Comparison with Existing Approaches

Product and library facts are limited to their public documentation, code or filings. Items seen only through a search-engine extract on 2026-10-08 are marked [S] in the References.

Table 4.1. Comparison with existing approaches.

Approach	Documented behavior	Overlap	Difference	Ref.
Function calling / tool use (OpenAI, Anthropic)	Model may call declared tools; output not checked	Tool layer (Section 3.2)	No absence taxonomy, number check or perimeter	[22] [20]
OpenAI Agents SDK (software development kit)	<code>function_tool</code> builds schema and dispatch from one declaration; <code>is_enabled</code> (2025-06-03) hides tools per run; <code>failure_error_function</code> and <code>timeout_behavior="error_as_result"</code> (2026-02-13) return failures as tool output	N1–N4 in general form	No key-pattern grammar, metadata-only range text, closed absence enumeration or per-signature miss limit	[23]
OpenHands stuck detector	Halts after 4 identical action–observation pairs or 3 identical erroring actions	N4	Emuqu counts only misses per signature per turn and returns an absence; it does not halt	[24]
RAG	Retrieved passages in context	Grounding	No output check	[4]
RARR / FActScore	Post-hoc claim check with model calls and retrieval	Verification	Model calls after generation	[7] [8]
Proof-Carrying Numbers (Sep 2025)	Claim-bound numbers verified against a source under a tolerance policy; fail-closed	N7	Uses claim-bound tokens; Emuqu parses free text and spelled numbers, scopes to present values, replaces only the span	[9]

Approach	Documented behavior	Overlap	Difference	Ref.
US Patent 12,670,149	Detects and corrects inaccurate numeric comparisons; re-prompts to fix	N7, N8	Present-value filtering, voice-chunk span replacement and consume-once feedback not described in the material seen	[10]
PHIA (Personal Health Insights Agent, 2024)	Agent answering wearable-data questions by generating and running code	Personal-data grounding	No post-generation number check or perimeter described	[25]
NeMo Guardrails	Programmable input/output rails; streaming output rails check chunks before the client	N12 timing	Server-side; no negation-neutral deflection or decimal-aware buffering documented	[16] [17]
Llama Guard / Llama Guard 3	Classifier over prompt and response; Llama Guard 3: 8 languages, S6 Specialized Advice, S11 Self-Harm	N10, N11	Model call per check; general taxonomy; no build-time link	[14] [15]
Myra crisis protocol	Every message screened before the AI (phrase list + classifier); fixed message plus region-localized crisis lines	N11	No ordered multi-class refusal, local-only history persistence or prompt-parity test documented	[26]
Keyword crisis gate (Hugging Face Space)	Fixed crisis message on a keyword match	N11	English keyword list	[27]
NegEx-style rules	Negation detection in clinical notes	— (not presented as new)	Emuqu does not use negation detection	[18]
exrex; Hypothesis from_regex	Generate strings matching a regex	N14 witness step	No cross-vocabulary coverage check or vacuity guards	[28] [29]
WHOOOP Coach	GPT-4 "fine-tuned with anonymized member data and proprietary WHOOOP algorithms"; zero retention	Health-data coach	Nothing public on number verification, guardrails or languages	[30]
Oura Advisor	LLM with Oura's algorithms; inputs include scores, contributors, tags, profile and past interactions; "not a medical device"; medical questions can be routed to Counsel Health's clinicians	Health-data coach; medical boundary	Nothing public on number verification or output filtering	[31] [32] [33]

Approach	Documented behavior	Overlap	Difference	Ref.
Emuqu (this paper)	Typed catalog with closed absence taxonomy; present-value verifier with span correction and next-turn feedback; one 17-language lexicon at input, output and build time	—	Regex on device, no model call; regulated-claim guard before display and speech; number check before speech (voice) and at round end (chat)	—

5. What Is New

Each statement gives only the part not found in a prior-art search made on 2026-10-08 (agent-framework code and documentation, patent databases, arXiv, vendor documentation, public code hosting). Commit dates are author dates in US Central time (CST = UTC−6, CDT = UTC−5). Each form of a statement was publicly disclosed when it reached the public repository: on 2026-09-08 (first commit 9d89933) for forms committed before that date, and on its commit date for later forms, unless an earlier public disclosure is noted. Statement numbering follows v1.1. The section after each label is where the method is specified.

N1 (Section 3.1.1). A fact catalog keyed by literal dotted keys or patterns with exactly one `$param`, parsed by one depth-aware grammar that keeps dots inside parentheses. The same parse yields the tool name (dots → underscores; (`$param`) → required string property), a leading-token match that passes remaining tokens as a sub-key, and dispatch (head-token namespaces first, in registration order, then all others), so a per-entry tool and a raw-key `lookup_fact` reach the same resolver.

Builds on: the OpenAI Agents SDK, which derives a tool's schema and dispatch from one declaration [23]; adds a dotted key-pattern grammar shared by naming, matching and dispatch.

Earliest dated record: flow-recovery c63d56c, 2026-04-23 (first form, "major AI voice upgrades"); emuqu-dev 680bbec, 2026-08-25 (current form, depth-aware split).

N2 (Section 3.1.4). A per-entry availability check that is synchronous and metadata-only (no HealthKit, files or network), returning `{hasData, validRange, lastUpdated}`. The schema builder drops entries without data and appends "Data available as early as ; do not request earlier dates" to each parameterized entry, giving only the month of the lower bound. The filtered schema is hashed (SHA-256, sorted-key JSON) to detect drift in cache-relevant bytes.

Builds on: the OpenAI Agents SDK `is_enabled` option, which hides a tool per run [23]; adds the metadata-only constraint and month-granular range text.

Earliest dated record: flow-recovery c63d56c, 2026-04-23 (first form); the same commit carries the current form.

N3 (Sections 3.1.2–3.1.3). A tool-result envelope whose absence channel is a closed nine-value enumeration with a defined meaning per value. `missingReason` is omitted on success, so its presence is the signal; nested

`missing` values are serialized as inline envelopes, reporting absence per field; one serializer feeds five hosted providers and the on-device dispatcher.

Builds on: the OpenAI Agents SDK, which returns tool exceptions and timeouts to the model as tool output [23]; adds a closed absence taxonomy with per-field nesting across six providers.

Earliest dated record: flow-recovery c63d56c, 2026-04-23 (first form, nine-case taxonomy identical); flow-recovery e36fee4, 2026-05-06 (current form, on-device path).

N4 (Section 3.1.6). A registry gate that steers the model through absences rather than halting the run. (a) A per-turn counter on the exact (`tool name`, `arguments JSON`) signature, incremented only by `missing` results, turns the third identical miss into a `rateLimited` absence telling the model to stop retrying. (b) A non-missing envelope over 80,000 UTF-8 bytes becomes `tooMuchData`. (c) Numeric arguments declared once with a closed range, optional excluded lower bound and integer flag reject out-of-range, non-finite, non-integral and wrong-field-count input (empty fields counted) as `invalidParameter`, stating the accepted range in words. The wall-clock budget (Section 3.1.6, rail 3) is not presented as new.

Builds on: per-tool timeouts returned as results (OpenAI Agents SDK [23]) and the OpenHands stuck detector, which halts after four repeated action–observation pairs or three repeated erroring actions [24]; adds miss-only, per-signature steering through an absence, with size and range rails in the same envelope.

Earliest dated record: flow-recovery c63d56c, 2026-04-23 (first form, miss counter and 80,000-byte cap); emuqu c1d875e, 2026-10-05 (current form, range-checked arguments).

N5 (Section 3.1.5). Composite facts that declare atomic dependencies, awaited into a registry-held prefetch map that a synchronous body reads and that is restored before anything can suspend. The composite returns a `{present, missing}` record with per-child absence reasons and `status: "partialData"` when any child is missing.

Builds on: dependency-declared computation in general; no specific public prior art was found. Adds scoped prefetch for a synchronous body and per-child attribution returned to the model.

Earliest dated record: flow-recovery c63d56c, 2026-04-23 (first form, present/missing records); emuqu 554540f, 2026-10-03 (current form, scoped prefetch).

N6 (Section 3.2). A two-tier tool surface: 21 fixed polymorphic read tools plus 18 allow-listed actions taken from the availability-filtered catalog under their original names. The list is deduplicated by name (first wins) and sorted for byte stability, and a system-prompt capability index is generated from the same read-tool list (first sentence, capped at 170 characters).

Builds on: tool calling [5][6][22] and BM25 ranking [21]; no specific prior art was found for the two-tier surface. Adds the fixed read tier over a catalog, audit-preserving action names and a generated index.

Earliest dated record: flow-recovery d5d6180, 2026-05-06 (first form, compact router); emuqu-dev 680bbee, 2026-08-25 (current form, deduplication and sorting). The capability index is present at emuqu-dev d4e5cf8, 2026-08-16, a squashed import, so that date is an upper bound.

N7 (Section 3.3). A free-text numeric verifier for present-value claims. It (a) checks only present-value claims, discarding a match whose sentence has a time or target/comparison marker or whose number is followed by a time or count unit; (b) normalizes English spelled numbers (0–99, one decimal, optional "negative") and maps offsets back, snapping to rewritten spans so "negative thirty-one point seven" is replaced whole; (c) replaces only the number-and-unit span; (d) checks recovery on both scales (a claim ≤ 10 within 1.0 of the 0–10 score is accepted); (e) sets sleep-duration tolerance by claim precision (5 min, or 30 min for whole hours); and (f) runs on each voice chunk before speech and at every chat tool-loop round end. Tolerance checks against structured values and correction before the reply is returned are not presented as new.

Builds on: Proof-Carrying Numbers [9] (claim-bound tokens verified under a tolerance policy, fail-closed); US Patent 12,670,149 [10] (detecting and correcting inaccurate numeric comparisons); PHIA [25] (code-computed answers from wearable data). Adds spelled-number parsing in free speech, present-value scoping by sentence markers and span-only replacement before speech.

Earliest dated record: flow-recovery 1b5ad2a, 2026-05-02 (first form, "hallucination guard": live voice chunks, tolerances, span replacement); emuqu 554540f, 2026-10-03 (current form, time and target markers). Intermediate: app-state claims flow-recovery 10fc926, 2026-05-17; spelled numbers flow-recovery e8ca54a, 2026-06-02.

N8 (Section 3.3.4). A correction loop that feeds discrepancies into the next turn's prompt. Discrepancies are deduplicated by (metric, claimed, actual), capped at four, and consumed exactly once into a variable section of the next turn's prompt outside the cached prefix, telling the model the user may have seen the wrong number and to call the named tool; the reply is not regenerated.

Builds on: US Patent 12,670,149, which re-prompts to fix a detected numeric error [10]; adds consume-once, deduplicated next-turn feedback outside the cache.

Earliest dated record: flow-recovery 68f32e5, 2026-05-08 (first form, "hallucination feedback": cap 4, consume once); emuqu 5c4a23d, 2026-10-04 (current form, deduplication).

N9 (Section 3.4). Structural resolution of user-versus-cache conflicts by prompt construction. Statements (not questions) in the user turns among the last six turns, excluding local-only turns, that assert metric values inject a user-stated block. Statements that contradict the cache or request an override also remove the cached training-load block, instead of instructing the model to prefer the user.

Builds on: the knowledge-conflict literature on model behavior [2]; adds resolution by prompt construction.

Earliest dated record: flow-recovery e8ca54a, 2026-06-02 (first form); the same commit carries the current form.

N10 (Section 3.5). One regulated-claim lexicon of 29 concepts in 17 languages, split into word-bounded Latin-script and Cyrillic alternatives and substring CJK and Arabic alternatives. Per-language, position-aware lookarounds exclude benign homonyms (e.g. sinus arrhythmia in each script), and the lexicon is consumed through three named groups (input refusal, output scrubbing, never-in-static-copy) with per-concept membership.

Builds on: Llama Guard 3, a multilingual safety classifier with Specialized Advice and Self-Harm categories [15]; adds a shared regex lexicon with homonym exclusions driving three enforcement points.

Earliest dated record: flow-recovery 69ee32d, 2026-04-26 and flow-recovery 0a3972d, 2026-05-03 (first form, English-only precursors); emuqu-dev 8c3999d, 2026-08-26 (current form).

N11 (Section 3.6). An input guard that answers locally, in a fixed order, and keeps refused turns off every provider. It (a) classifies in the order self-harm, emergency, injury, rhythm, danger judgement; (b) answers with a fixed catalog-localized reply, and unit tests show that the cloud and on-device prompts quote the symptom and injury replies verbatim; (c) persists both turns flagged `localOnly` on the stored turn, honored by history assembly (before budgeting), summarization and the user-assertion detector; (d) sends nothing to any provider and runs before consent; and (e) answers self-harm even with its feature flag off.

Builds on: Myra's crisis protocol, which screens every message before the AI with a phrase list and classifier and replies with a fixed message and region-localized crisis lines [26], and a public Hugging Face Space with a keyword crisis reply [27]. Adds ordered multi-class refusal, local-only persistence across history and tested prompt parity.

Earliest dated record: flow-recovery 69ee32d, 2026-04-26 (first form, English, nothing sent); emuqu c1d875e, 2026-10-05 (current form, injury class). Intermediate: `localOnly` flow-recovery 9645baa, 2026-05-01; ordered classes emuqu-dev 8c3999d, 2026-08-26; self-harm class emuqu 3585687, 2026-09-30.

N12 (Section 3.7). An on-device, negation-neutral output guard that replaces whole sentences with deflections true in both readings. It retries on markdown-stripped text; collapses a sentence-straddling match into one deflection, or deflects the whole message if that still matches; and runs on complete sentences before display (holding back tails with a decimal-aware split), on every spoken utterance, and in a final pass.

Builds on: NeMo Guardrails streaming output rails, server-side [17], and NegEx [18]. Adds negation-neutral sentence deflection on the device before display and speech.

Earliest dated record: flow-recovery 0a3972d, 2026-05-03 (first form, English sentence deflection); emuqu 5c4a23d, 2026-10-04 (current form, decimal-aware split). Intermediate: negation-neutral multilingual form emuqu-dev 8c3999d, 2026-08-26.

N13 (Section 3.8). A build-time copy linter with a Swift lexical scanner (nested comments, multi-line and raw literals, blanked interpolations). It evaluates a prohibition exemption on comment-stripped source within the prohibiting sentence only, allow-lists at the key level so translations are covered, applies multilingual patterns parsed from the runtime lexicon source to every locale, and fails closed on bad input.

Builds on: regular-expression keyword filtering; no specific prior art was found. Adds the scanner, sentence-scoped exemption and lexicon-sourced multilingual check.

Earliest dated record: flow-recovery 0a3972d, 2026-05-03 (first form: Swift string scanner, comment stripping, string catalogs); emuqu-dev 8c3999d, 2026-08-26 (current form, multilingual patterns). Intermediate: prohibition context emuqu-dev 59ce5da, 2026-08-26.

N14 (Section 3.9). A coverage gate that synthesizes a witness for each build-time regex, verifies it against its own pattern, and requires a runtime-lexicon alternative to match it. It refuses to pass when any alternative matches the empty string or fewer than one hundred were extracted.

Builds on: regex string generation (exrex [28]; Hypothesis `from_regex` [29]); adds its use to check that one vocabulary covers another, with self-check and vacuity guards.

Earliest dated record: emuqu-dev 8c3999d, 2026-08-26 (first form); the same commit carries the current form.

Prior work used, not presented as new.

- Model tool or function calling [5][6][22].
- Deriving tool schema, dispatch and enablement from one declaration, and returning tool errors and timeouts to the model as results [23].
- Per-call wall-clock budgets.
- Loop detection on repeated agent actions [24].
- Retrieval-augmented generation [4] and provider-side prompt caching [20].
- BM25 ranking [21].
- Regular-expression keyword filtering in general.
- Classifier- or rail-based LLM guardrails, including multilingual classifiers and streaming output rails checked before delivery [14][16][15][17].
- Pre-model crisis screening with a fixed reply [26][27].
- Verification or correction of model numbers against structured values with tolerances, correction before the reply is returned, and re-prompting on an error [9][10].
- Post-hoc claim verification with model calls [7][8].
- Clinical negation detection [18].
- Generating strings from a regular expression [28][29].
- Multi-provider fallback and per-vendor consent prompts.
- Instruction hierarchies [13][34].

6. Verification

These methods are software controls, not physiological measurements. The science register (`Tools/science_register/register.json`, scoring version v3.1.oct2026) classifies the app's physiological heuristics; none of its entries covers a method in this paper. Test results for the physiological methods of Paper 1, *Overnight Recovery Measurement from Raw Beat Intervals* ([link](#)), and Paper 2, *Real-Time DFA α_1 and Dropout-Robust Training Load* ([link](#)), are in the cross-implementation report, `Tools/validation/RESULTS.md`. Verifier tolerances are set from dashboard rounding and metric noise; detection rates on real model output are not claimed.

Behavior is fixed by unit tests in `EmuquTests` (Table 6.1; test-function counts at e028039) and by CI gates.

Table 6.1. Unit-test suites.

Suite	Tests	Establishes
EmuquTests/CoachVoiceGuardTests.swift	44	200 identical prohibited sentences all rewritten; 100 clean sentences byte-identical among 100 prohibited; "you may have atrial fibrillation" intercepted in all 16 non-English locales; sinus arrhythmia passes in every locale; straddles fail closed; decimals do not split; nothing matched is published mid-stream
EmuquTests/CoachVoiceGuardNegationTests.swift	4	Denial and assertion of a term get the same neutral line; every translated deflection exists and passes
EmuquTests/MedicalQueryGuardTests.swift	29	Rhythm and symptom questions refused in every locale; crisis reply for self-harm in every locale; "suicide sprints" proceed; symptom outranks rhythm; rule B quotes the reply verbatim; every <code>refuseBeforeSending</code> concept refused, with lexicon-derived probes
EmuquTests/MedicalQueryGuardTests+Injury.swift	6	Injury class and reply
EmuquTests/LocalOnlyTurnTests.swift	7	<code>localOnly</code> persists; refused turns withheld from history, not counted as dropped, also under truncation
EmuquTests/HallucinationFeedbackTests.swift	15	Buffer caps at four, clears on consume; wrong current HR, duration, efficiency corrected without the label; targets, norms, other nights, averages ignored; spelled durations checked; right or rounded values kept
EmuquTests/FactValueTests.swift	41	No <code>missingReason</code> on success; non-finite nulls only its field; nested missing envelopes; sorted keys; valid JSON
EmuquTests/FactCatalogValidationTests.swift	8	Unique, provider-valid tool names; valid placeholders; real composite dependencies; fast availability; deterministic schema; every key round-trips
EmuquTests/FactNumericArgumentTests.swift	9	Hostile numeric values and field counts rejected with the range named, without trapping

`Tools/copy_linter/lint.py` and `scripts/check_perimeter_sync.sh` run in CI.

`scripts/verify_gates_fail.sh` plants a violation for each and requires failure: a localized "This can diagnose your arrhythmia." and a malformed catalog for the linter, and a term added to only one list for the coverage gate. At e028039 the coverage gate probes 62 patterns against 985 alternatives, with no gaps.

7. Design Tradeoffs

- The perimeter is lexical by choice (deterministic, on-device, auditable), so a paraphrase that avoids the vocabulary is not caught, and obfuscation beyond removing emphasis markers is not normalized.
 - Negation-neutral deflection buys correctness in all 17 languages at the cost of also deflecting correct denials ("there's no sign of overtraining"), and a sentence with a matched term is replaced whole.
 - The guards take precedence over prompt rules that permit general descriptions of conditions, so the coach does not deliver general information about named conditions.
 - The input guard reads the current message only, keeping its decision immediate; a question split across turns is not joined.
 - Number verification covers present-value claims for the 11 metrics in Table 3.4, in English, so historical values, averages and non-English replies are not corrected; live-workout claims are checked only on the voice path.
 - In chat the correction lands at the end of the round, after its text may have been displayed; the next-turn reminder addresses that.
 - The 21 compact read tools are static for byte stability, so availability gating applies to the per-entry schema and the action tools.
 - A round that would exceed the 8-call budget has all its calls refused, which keeps the rule simple and leaves the model one clean round to answer.
 - The coverage gate is one-directional: build-time terms must be covered at runtime, while the runtime lexicon may be broader.
 - Model-drafted email bodies go to the system mail composer for the user to review and send, and auto-extracted memories are written by a background model call; both sit outside the output guard.
-

8. Provenance and Dates

8.1 Project origin

The project began as a general fitness app. Repository AI-Fitness-Coach (2025-02-16 to 2025-02-17, 19 commits, private) displayed Apple HealthKit heart rate and HRV (SDNN), streamed Polar H10 heart rate over Bluetooth and estimated VO₂max; it contains no RR-interval analysis. On 2025-02-22 (15:44 UTC) the author proposed, in a ChatGPT conversation, an app that would take an HRV reading from a chest strap worn overnight just before the user wakes (excerpt recovered through conversation-history retrieval on 2026-10-08). That thread is separate from the 2025 records of the author's design-audit method, which concern another project (Paper 5). The Emuqu design itself begins with the full-night capture problem statement of December 2025 (Paper 5, Section 4).

8.2 Repository lineage

Table 8.1. Repository lineage. All repositories are owned by the author and retained with full history.

Repository	Product name	First commit	Extent	Visibility
chrisharp80/AI-Fitness-Coach	(fitness app)	2025-02-16	19 commits to 2025-02-17	Private
chrisharp80/flow-recovery-old	Flow HRV	e5948bd, 2026-01-12	250 commits to 2026-01-26	Private
chrisharp80/flow-recovery-dev	Flow Recovery	c48eeb9, 2026-03-01	Squashed import; to 2026-03-15	Private
chrisharp80/flow-recovery	Flow Recovery	39bfcd4, 2026-03-15	196 commits to 2026-08-12	Private
chrisharp80/emuqu-dev	Emuqu	d4e5cf8, 2026-08-16	Squashed import; to 2026-09-04	Private
chrisharp80/emuqu	Emuqu	9d89933, 2026-09-08	Squashed import; current	Public (PolyForm Strict 1.0.0)

History has gaps from 2026-01-26 to 2026-03-01, 2026-08-12 to 2026-08-16 and 2026-09-04 to 2026-09-08, where each new repository began with a squashed import; a date at a squashed import is an upper bound for anything it contains. The pre-code design (v1 to v9.2 FINAL, completed 2026-01-08 21:06 CST) is described in Paper 5, Section 4: versions v1 and v3.0 have not been recovered; v2–v2.2 and v4.0–v9.1 survive in the author's Claude conversation record; v9.2 FINAL survives as a Google Drive document.

8.3 Dated records outside git

Documents in the author's own Google Drive, dated 2026-05-06 and uploaded on 2026-05-10 (README 2.md, VOICE_AND_TOOL_USE 2.md, flow-recovery-audit.md, audit-results-20260506), describe the fact catalog as the tool schema (about 200 tools, 19 namespaces). They also describe:

- availability-closure gating with the valid range in parameter descriptions;
- a uniform `{value, missingReason, asOf, confidence}` envelope across Apple and cloud providers;
- the registry rails: the third identical miss becomes `rateLimited`, an 80,000-byte cap gives `tooMuchData`, and resolvers have a 2 s budget;
- composite `{present, missing}` records and `[ACTION]` tools;
- the input guard and the output regex guard;
- a copy-linter term list with the `(?<!sinus\s)` lookbehind on "arrhythmia".

These agree with the commit dates in Section 5.

8.4 Public disclosures

- 2026-01-27: TestFlight beta opened (over 1,500 sessions by 2026-03-06; 45 testers and over 6,500 sessions by August 2026). A public TestFlight join link appeared in the project README by May 2026.
- 2026-05-19: Substack article "A Powerfully Lazy Man's Way to Better Health" (Paper 1 lists what it disclosed).

- 2026-08-10: Substack article "How I Build Production Software by Directing AI" (Paper 5 lists what it disclosed).
- 2026-09-08: public repository github.com/chrissharp80/emuqu (first commit 9d89933).
- 2026-10-08: these white papers.

Neither Substack article disclosed any method in this paper.

8.5 Authorship

Chris Sharp is the sole author of these papers and of the design, and the only person who commits to the repositories above. The code was written by AI coding assistants under his direction, and some commits carry "Claude" as author or co-author; he supplied the requirements, design direction, review, testing and acceptance, and wrote no code by hand. He developed the methods described here independently and did not learn them from anyone.

8.6 Document history

v1.0, v1.1, v1.2 and v1.3 all published 2026-10-08. v1.1 added prior-art comparison and provenance; v1.2 restructured method-first and moved verification detail to the cross-implementation report; v1.3 applied an editorial review (series-wide format, corrected internal references, tightened claims). This paper (v1.3) describes source revision e028039 (2026-10-07).

References

Items marked [S] were seen only through a search-engine extract on 2026-10-08.

1. Ji Z, Lee N, Frieske R, et al. Survey of hallucination in natural language generation. *ACM Comput Surv.* 2023;55(12).
2. Xu R, Qi Z, Guo Z, Wang C, Wang H, Zhang Y, Xu W. Knowledge conflicts for LLMs: a survey. In: *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. 2024. p. 8541–8565. doi:10.18653/v1/2024.emnlp-main.486
3. Vu T, Iyyer M, et al. FreshLLMs: refreshing large language models with search engine augmentation. In: *Findings of the Association for Computational Linguistics: ACL 2024*. 2024. p. 13697–13720. arXiv:2310.03214.
4. Lewis P, Perez E, Piktus A, et al. Retrieval-augmented generation for knowledge-intensive NLP tasks. *Adv Neural Inf Process Syst.* 2020;33.
5. Schick T, Dwivedi-Yu J, Dessì R, et al. Toolformer: language models can teach themselves to use tools. *Adv Neural Inf Process Syst.* 2023;36.
6. Yao S, Zhao J, Yu D, et al. ReAct: synergizing reasoning and acting in language models. In: *International Conference on Learning Representations (ICLR)*. 2023.
7. Gao L, Dai Z, Pasupat P, et al. RARR: researching and revising what language models say, using language models. In: *Proceedings of ACL 2023*. 2023.

8. Min S, Krishna K, Lyu X, et al. FActScore: fine-grained atomic evaluation of factual precision in long form text generation. In: Proceedings of EMNLP 2023. 2023.
9. Solatorio. Proof-Carrying Numbers (PCN): a protocol for trustworthy numeric answers from LLMs via claim verification [Internet]. arXiv:2509.06902. 2025. <https://arxiv.org/abs/2509.06902> (accessed 2026-10-08) [S]
10. United States Patent and Trademark Office. US Patent 12,670,149: Detection and correction of inaccurate numeric comparisons in LLM responses [Internet]. <https://image-ppubs.uspto.gov/dirsearch-public/print/downloadPdf/12670149> (accessed 2026-10-08) [S]
11. US Food and Drug Administration. General wellness: policy for low risk devices. Guidance for industry and Food and Drug Administration staff [Internet]. 2026 Jan 6 (supersedes the 2019 version). <https://www.fda.gov/media/90652/download> (accessed 2026-10-08) [S]
12. Apple Inc. App Store Review Guidelines, Guideline 1.4.1 [Internet]. <https://developer.apple.com/app-store/review/guidelines/> (accessed 2026-10-08)
13. Greshake K, Abdelnabi S, Mishra S, et al. Not what you've signed up for: compromising real-world LLM-integrated applications with indirect prompt injection. In: Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security (AISec). 2023.
14. Inan H, Upasani K, Chi J, et al. Llama Guard: LLM-based input-output safeguard for human-AI conversations [Internet]. arXiv:2312.06674. 2023. <https://arxiv.org/abs/2312.06674> (accessed 2026-10-08)
15. Meta. Llama Guard 3 model card [Internet]. 2024. <https://huggingface.co/meta-llama/Llama-Guard-3-8B> (accessed 2026-10-08)
16. Rebedea T, Dinu R, Sreedhar M, Parisien C, Cohen J. NeMo Guardrails: a toolkit for controllable and safe LLM applications with programmable rails. In: Proceedings of EMNLP 2023: System Demonstrations. 2023.
17. NVIDIA. NeMo Guardrails documentation: streaming and output rails [Internet]. <https://docs.nvidia.com/nemo/guardrails/> (accessed 2026-10-08)
18. Chapman WW, Bridewell W, Hanbury P, Cooper GF, Buchanan BG. A simple algorithm for identifying negated findings and diseases in discharge summaries. J Biomed Inform. 2001;34(5).
19. Chapman WW, Hillert D, Velupillai S, Kvist M, Skeppstedt M, et al. Extending the NegEx lexicon for multiple languages. Stud Health Technol Inform. 2013;192:677–681. doi:10.3233/978-1-61499-289-9-677
20. Anthropic. Tool use and prompt caching. Anthropic API documentation [Internet]. 2024. <https://docs.anthropic.com/en/docs/build-with-claude/prompt-caching> (accessed 2026-10-08) [S]
21. Robertson S, Zaragoza H. The probabilistic relevance framework: BM25 and beyond. Found Trends Inf Retr. 2009;3(4).
22. OpenAI. Function calling. OpenAI API documentation. 2023.
23. OpenAI. OpenAI Agents SDK for Python: function_tool, is_enabled (pull request #808, merged 2025-06-03), failure_error_function, and timeout_behavior="error_as_result" (added 2026-02-13), in src/agents/tool.py [Internet]. <https://github.com/openai/openai-agents-python> (accessed 2026-10-08)
24. OpenHands. Agent stuck detector [Internet]. <https://docs.openhands.dev/sdk/guides/agent-stuck-detector> (accessed 2026-10-08)
25. Merrill MA, Paruchuri A, Rezaei N, et al. Transforming wearable data into health insights using large language model agents (PHIA) [Internet]. arXiv:2406.06464. 2024. <https://arxiv.org/abs/2406.06464> (accessed 2026-10-08)
26. Myra. Myra — Crisis Response Protocol [Internet]. <https://sites.google.com/view/myra-privacy-policy/home/crisis-resource> (accessed 2026-10-08)

27. Deva1211. chatbot, Hugging Face Space, app.py [Internet]. <https://huggingface.co/spaces/Deva1211/chatbot/blob/main/app.py> (accessed 2026-10-08)

28. exrex: irregular methods on regular expressions (Python package) [Internet]. <https://pypi.org/project/exrex/> (accessed 2026-10-08)

29. Hypothesis documentation: hypothesis.strategies.from_regex [Internet]. <https://hypothesis.readthedocs.io/> (accessed 2026-10-08)

30. OpenAI. WHOOP customer story [Internet]. <https://openai.com/customer-stories/whoop> (accessed 2026-10-08) [S]

31. Oura. Oura Advisor (support article) [Internet]. <https://support.ouraring.com/hc/en-us/articles/39512345699219> (accessed 2026-10-08) [S]

32. Oura. Oura Advisor, an AI-powered personal health companion, now rolling out to all Oura members [Internet]. Business Wire. 2025 Mar 31. <https://www.businesswire.com/news/home/20250331565896/en/> (accessed 2026-10-08)

33. Oura. Counsel integration in the Oura app (blog) [Internet]. <https://ouraring.com/blog/counsel-integration-oura-app/> (accessed 2026-10-08)

34. Wallace E, Xiao K, Leike R, et al. The instruction hierarchy: training LLMs to prioritize privileged instructions [Internet]. arXiv:2404.13208. 2024. <https://arxiv.org/abs/2404.13208> (accessed 2026-10-08)

Appendix A: Parameters

Table A.1. Parameters at e028039.

Name	Value	Unit	Role
Registered namespace resolvers	26	count	Registry address space
Declared catalog entries	238 (149 fixed sync, 8 fixed async, 56 parameterized, 6 composite, 19 action)	count	Catalog size
Absence reasons	9	count	Closed taxonomy
<code>rateLimitMissRetries</code>	2	misses	Third identical miss per turn → <code>rateLimited</code>
<code>outputSizeCapBytes</code>	80,000	bytes	Per-result cap → <code>tooMuchData</code>
<code>defaultResolveBudgetSec</code>	2.0	s	Per-entry path: overrun discards value
<code>resolveWarnBudgetSec</code>	0.5	s	Slow-resolver warning
<code>watchdogWarnSec</code>	4.0	s	Action main-thread warning
Valid-range text granularity	month of lower bound	—	Parameter description
Compact read tools	21	count	Model-facing schema
Allow-listed action tools	18	count	Model-facing schema

Name	Value	Unit	Role
Capability-index purpose cap	170	characters	Per tool line
BM25 k1, b	1.5, 0.75	—	Tool ranking
BM25 target K	40	tools	Ranker filters only above this
xAI Grok schema cap	110	tools	Per-provider cap; does not bind at 39 tools
<code>maxToolCallsPerTurn</code>	8	calls	Over-budget round → all calls <code>rateLimited</code>
Stream publish interval	33	ms	Throttle; sentence-complete publishing
HR tolerance	5	bpm	Verifier
Power tolerance	15	W	Verifier
DFA α 1 tolerance	0.10	—	Verifier
HR drift tolerance	1.5	percentage points	Verifier
TSB tolerance	2.0	load units	Verifier
ATL / CTL tolerance	3.0	load units	Verifier
ACWR tolerance	0.15	ratio	Verifier
RMSSD tolerance	8.0	ms	Verifier
Recovery tolerance	4.0 (0–100) / 1.0 (0–10)	points	Verifier, scale disambiguation
Sleep duration tolerance	5 (precise) / 30 (whole hours)	min	Verifier
Sleep efficiency tolerance	1.0	percentage points	Verifier
Spelled-number range	0–99, one decimal digit	—	Normalizer
Pending corrections cap	4	entries	Next-turn reminder
User-assertion detector window	last 6 turns, filtered to non-local user turns	turns	Assertion detection
Numeric arg: <code>starting_atl</code> , <code>starting_ctl</code> , <code>daily_trimp</code>	0–10,000	TRIMP (training impulse)	Range check
Numeric arg: <code>gap_trimp</code>	(0, 10,000]	TRIMP	Range check
Numeric arg: <code>horizon_days</code>	1–365, integer	days	Range check
Numeric arg: <code>lat</code> / <code>lon</code>	–90–90 / –180–180	degrees	Range check
Numeric arg: <code>radius_m</code>	5–5,000	m	Range check
Numeric arg: <code>ordinal</code>	0–100,000, integer	index	Range check
Numeric arg: <code>seconds</code>	1–86,400, integer	s	Range check

Name	Value	Unit	Role
Numeric arg: <code>max_results</code>	1–10, integer	results	Range check
Lexicon concepts	29 (+2 offensive)	count	Shared vocabulary
Languages	17 shipped locales (en + 16)	count	Lexicon and replies
<code>refuseBeforeSending</code> / <code>scrubFromOutput</code> / <code>neverInStaticCopy</code>	12 / 21 / 12	concepts	Group sizes
Sentence terminators	<code>. ! ? \n . ! ? ,</code> <code>¿ ...</code>	—	Segmentation
Emphasis markers stripped on retry	<code>* _ `</code>	—	Output guard
Offensive-language recognizer confidence	0.6	probability	English-only entries
Offensive-language minimum words	3	words	English-only entries
Voice echo overlap	0.40	token fraction	Refusal echo trap
Linters prohibition window	3 lines (match line and 2 above)	lines	Exemption
Linters snippet fragment	first 40 characters, at least 3	characters	Exemption
Build-time English patterns	62	count	<code>user_facing_prohibited</code>
Coverage-gate minimum alternatives	100 (985 at e028039)	count	Vacuity guard
Consent schema version	8	—	Per-provider consent
Request audit size	10	entries	In-memory first-in-first-out (FIFO) log

Context: consent and provider fallback (not presented as new). Each hosted provider needs its own consent before its first send. The consent key includes a schema version (currently 8) that is bumped when the disclosure changes, and removing an application programming interface (API) key revokes consent. The on-device provider is exempt. On failure, the fallback chain holds only other enabled, available, consented hosted providers, then the on-device provider. When the on-device model refuses a request on safety grounds, the refusal is shown and the request is not retried with another provider. Each outbound request is recorded, before provider-specific transforms, in an in-memory FIFO log of 10 entries.

Appendix B: Pseudocode

B.1 Registry tool resolution

```

resolveTool(name, argsJSON):
  entry = findEntry(toolName == name) or return missing(invalidParameter, "unknown tool")
  if entry is action:
    args = parse(argsJSON) keeping only declared params, numbers/bools -> strings
    for p in entry.params where p.required and p not in args:
      return gate(name, argsJSON, missing(invalidParameter, "missing required arg
p"))
    v = await entry.execute(args)
    return gate(name, argsJSON, v)
  key = entry.key, or entry.pattern with "$param" replaced by args[param]
    (missing/bad arg -> missing(invalidParameter))
  t0 = now; v = await resolveAsync(key); dt = now - t0 # range checks run inside the
resolver
  if dt > 2.0: v = missing(internalError, "timed out")
  return gate(name, argsJSON, v)

gate(name, argsJSON, v):
  if v is missing:
    sig = name + ":@" + argsJSON; count[sig] += 1
    return count[sig] > 2 ? missing(rateLimited, ...) : v
  if utf8len(envelope(v)) > 80000: return missing(tooMuchData, ...)
  return v

```

B.2 Composite (async path)

```

compositeAsync(entry, key):
  param = entry.key has "(" ? capture(entry.key, key) : nil
  deps = entry.dependencies with "$name" -> param
  values = { d: await resolveAsync(d) for d in unique(deps) }
  saved = prefetch; prefetch = prefetch u values
  r = entry.body(param, registry) # body's registry.resolve(k) reads prefetch
first
  prefetch = saved
  return r # {present:[{key,value}], missing:
[{"key,reason,detail"}], status?}

```

B.3 App-state number verification

```

verifyAppState(text):
    (norm, rewrites) = normalizeSpelledNumbers(text)
    found = []
    for (pattern, actual, tol, fmt, negateOnWord) in metricChecks:
        for m in pattern.matches(norm):
            s = sentenceAround(m, norm)
            if s matches timeMarkers or targetMarkers: continue
            c = number(m.value); if negateOnWord and m.claim contains "negative": c = -|c|
            if |c - actual| > tol: found += (metric, fmt(c), fmt(actual), m.claimRange)
    apply recovery-scale rule and sleep sentence filters
    mapped = [remap(d.range, norm -> text, rewrites) for d in found]
    corrected = replace mapped ranges back-to-front in text with d.actual
    recordCorrections(mapped)                # deduplicate (metric, claimed, actual); keep
last 4
    return corrected

nextPromptVariableSections():
    block = takeAndClear(pendingCorrections)
    if block nonempty: append "# Last turn correction – DO NOT FABRICATE" + one line per
entry

```

B.4 User-assertion detector

```

signals = {}
for msg in reversed(lastSixTurns.filter(user and not localOnly)):
    t = statementsOnly(normalizeSpelledNumbers(msg))
    for metric in knownMetricsLongestFirst:
        v = lastMatch(metric, connector required if metric in {sleep, recovery}, no clock
times)
        if v and metric not in signals.values: signals.values[metric] = v; blank matches
        fold aliases into canonical if canonical absent
    signals.contradicted |= containsPhrase(t) or startsWithCorrectionReply(t)
    signals.override     |= containsOverridePhrase(t)
if signals nonempty: prompt += userStatedBlock(signals)
if not (signals.contradicted or signals.override): prompt += dashboardLoadBlock

```

B.5 Input guard

```

onSend(text):
    t = trim(text); if empty: reject
    trig = first of [selfHarm, emergency, injury, rhythm, dangerJudgement] whose regexes
match t
    if trig and (flagOn or trig == selfHarm):
        store(userTurn(t, localOnly=true), assistantTurn(replyFor(trig, appLanguage),
localOnly=true))
        return                                # no provider, no consent prompt
    continue to consent check and provider send
history(turns) = turns.filter(not localOnly and not emptyAssistant)

```

B.6 Streaming output guard

```
onDelta(chunk):
  buf += chunk
  if now - lastPublish < 33ms: return
  (complete, tail) = splitAtLastTerminator(buf)  # decimal-aware
  if complete empty: return
  buf = tail
  visible += scrub(complete)
  if roundHasNoToolCall and published text has terminator: speakableCursor = len(visible)
onRoundEnd / onError / onStop:
  visible += scrub(buf); buf = ""
speak(utterance):
  synthesize(preamble + verify(scrub(utterance)))
```

B.7 Coverage gate

```
alts = [compile(latin ? "\b(?:\"+a+)\b" : a) for a in literals(latin/unbounded arrays of
lexicon*)]
fail(2) if any alt matches "" or len(alts) < 100
for p in buildTimePatterns:
  w = witness(p)                                # see Section 3.9, step 3
  if w == "": continue                          # skipped, not counted; none at e028039
  fail(2) if not p matches w
  gaps += p if no alt matches w
fail(1) if gaps else pass
```

Appendix C: Source Map

Table C.1. Source map (repository-relative paths).

Method	Files
Entry kinds, namespace walk, pattern matching	Emuqu/Sources/Assistant/Facts/FactCatalog.swift
Key grammar	Emuqu/Sources/Assistant/Facts/FactKey.swift
Value union, absence taxonomy, availability, envelope	Emuqu/Sources/Assistant/Facts/FactValue.swift
Registry, rails, schema, hash, composites, time budget	Emuqu/Sources/Assistant/Facts/FactResolverRegistry.swift
Numeric argument ranges	Emuqu/Sources/Assistant/Facts/AppFactResolver+NumericArguments.swift
Namespace registration; entries	Emuqu/Sources/Assistant/Facts/AppFactResolver.swift and the AppFactResolver+ extensions beside it

Method	Files
Provenance fields (<code>data_source</code> , <code>comparison_safe</code>)	<code>Emuqu/Sources/Assistant/Facts/AppFactResolver+TrainingProjections.swift</code> , <code>Emuqu/Sources/Assistant/Facts/AppFactResolver+WorkoutLiveDerived.swift</code> , <code>Emuqu/Sources/Assistant/Facts/AppFactResolver+Sleep.swift</code>
Compact tools, allow- list, capability index	<code>Emuqu/Sources/Assistant/Facts/CompactToolRouter.swift</code> , <code>Emuqu/Sources/Assistant/Facts/CompactToolRouter+Resolvers.swift</code>
BM25 ranking	<code>Emuqu/Sources/Assistant/Facts/ToolRetriever.swift</code>
xAI Grok schema cap	<code>Emuqu/Sources/Assistant/Providers/GrokProvider.swift</code>
Tool loop, call budget, chat verification, fallback	<code>Emuqu/Sources/Assistant/ViewModel/AssistantViewModel+Tools.swift</code>
On-device dispatcher	<code>Emuqu/Sources/Assistant/Providers/AppleToolDispatcher.swift</code>
Number verifier, correction buffer	<code>Emuqu/Sources/Assistant/MetricsVerifier.swift</code>
Voice verification and scrub order	<code>Emuqu/Sources/Assistant/VoiceConversationController+Speech.swift</code>
User-assertion detector	<code>Emuqu/Sources/Assistant/UserAssertedValuesParser.swift</code>
Prompt composition (cache suppression, corrections)	<code>Emuqu/Sources/Assistant/Providers/AIProvider+SystemPrompt.swift</code> ; rules A–D in <code>Emuqu/Sources/Assistant/Providers/AIProvider+SystemPromptText.swift</code>
Lexicon and groups	<code>Emuqu/Sources/Assistant/MedicalTermLexicon.swift</code> , <code>Emuqu/Sources/Assistant/MedicalTermLexicon+Injury.swift</code> , <code>Emuqu/Sources/Assistant/MedicalTermLexicon+MetricVerdict.swift</code>
Input guard	<code>Emuqu/Sources/Assistant/MedicalQueryGuard.swift</code> ; call site <code>Emuqu/Sources/Assistant/ViewModel/AssistantViewModel.swift</code> ; echo trap <code>Emuqu/Sources/Assistant/VoiceConversationController+Audio.swift</code>
<code>localOnly</code> turns	<code>Emuqu/Sources/Assistant/Providers/AIProvider.swift</code> (ChatTurn); <code>Emuqu/Sources/Assistant/ViewModel/AssistantViewModel+Context.swift</code>
Output guard, offensive-language lists	<code>Emuqu/Sources/Assistant/CoachVoiceGuard.swift</code>
Streaming buffer	<code>Emuqu/Sources/Assistant/ViewModel/AssistantViewModel+Context.swift</code> (<code>StreamTextBuffer</code>)
Final pass	<code>Emuqu/Sources/Assistant/ViewModel/AssistantViewModel+Routing.swift</code>
Copy linter and configuration	<code>Tools/copy_linter/lint.py</code> , <code>Tools/copy_linter/prohibited_terms.json</code>
Coverage gate; gate failure check	<code>scripts/check_perimeter_sync.sh</code> ; <code>scripts/verify_gates_fail.sh</code>
Consent	<code>Emuqu/Sources/Assistant/ProviderConsentTracker.swift</code>
Tests cited	the nine suites in Table 6.1

Method	Files
Science register	Tools/science_register/register.json